

LECTURE 15

Classical Synchronization Problems I

CSC323 – Operating Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- The bounded-buffer problem's setup, and why it needs three separate semaphores rather than one
- The full producer and consumer pseudocode, built from wait()/signal()
- Why the order of acquiring semaphores is not a stylistic choice — getting it backward deadlocks the system
- The readers-writers problem's setup, and the “first” readers-writers variant's reader-preference behavior (and the starvation it risks)
- The semaphore-based solution, using a protected read_count alongside a rw_mutex

The Bounded-Buffer (Producer-Consumer) Problem

- A fixed-size buffer holds up to N items. One or more producer processes generate items and insert them; one or more consumer processes remove items and use them.
- A producer must not insert into a buffer that is already full.
- A consumer must not remove from a buffer that is already empty.
- Two processes manipulating the buffer's internal bookkeeping at the same instant can corrupt it — exactly Lecture 13's race condition, now with a circular buffer's index.

Three Semaphores, Three Separate Jobs

empty

Counting semaphore, init N — counts free slots.

full

Counting semaphore, init 0 — counts filled slots.

mutex

Binary semaphore, init 1 — protects the buffer index itself.

The Producer

```
semaphore empty = N;    /* free slots      */
semaphore full  = 0;    /* filled slots   */
semaphore mutex = 1;    /* buffer index lock */

/* Producer */
do {
    /* produce an item into next_produced */
    wait(empty);
    wait(mutex);
        /* add next_produced to the buffer */
    signal(mutex);
    signal(full);
} while (true);
```

The Consumer

```
/* Consumer */
do {
    wait(full);
    wait(mutex);
    /* remove an item from the buffer into next_consumed */
    signal(mutex);
    signal(empty);
    /* consume the item in next_consumed */
} while (true);
```

Why the Order Matters

- Each process waits on the counting semaphore first (empty for the producer, full for the consumer) and only then acquires mutex to touch the buffer's shared index.
- On the way out, it releases mutex first and signals the counting semaphore second.
- The counting semaphore is never acquired while holding mutex — this ordering is what keeps the system deadlock-free.

The Wrong Order

```
/* WRONG order */  
wait(mutex);  
wait(empty);  
    /* add item */  
signal(empty);  
signal(mutex);
```

Swap the Order and the System Deadlocks

SWAP THE ORDER AND THE SYSTEM DEADLOCKS

Suppose the buffer is completely full ($\text{empty} == 0$). The producer acquires mutex successfully, then calls `wait(empty)` — which blocks, because there are no free slots. Critically, the producer is now blocked while still holding mutex.

The consumer, which needs mutex to remove an item and eventually call `signal(empty)`, can never acquire it — mutex is held by a process that is never going to release it.

Both processes wait forever, each for something only the other could have provided. This is a deadlock, caused entirely by acquiring the two semaphores in the wrong order.

Hand-Tracing the Correct Version

- If the buffer is full, the producer blocks on `wait(empty)` without mutex; the consumer can freely acquire mutex, remove an item, release mutex, and call `signal(empty)`, which wakes the waiting producer.
- The symmetric case (empty buffer, consumer blocked on `wait(full)`) resolves the same way.
- Neither process is ever stuck holding mutex while waiting on a counting semaphore that only the other process can signal.

The Readers-Writers Problem

- A shared data object — a file, a database record, an in-memory table — is accessed by two kinds of processes.
- Readers only read the data and never modify it; writers modify it (and may read it too).
- Multiple readers may safely access the object simultaneously, since reading in parallel causes no conflict between them.
- A writer, however, needs exclusive access: no other reader or writer may touch the object while a writer is writing.

Shared Access for Readers, Exclusive for a Writer

Readers (any number)

May all access the shared data at the same time.

Writer (at most one)

Needs the shared data completely to itself — no readers, no other writers.

The “First” Readers-Writers Problem

- This lecture covers the first readers-writers problem: no reader should be kept waiting unless a writer has already obtained permission to use the shared object.
- Readers are given priority — as long as at least one reader is active, every newly arriving reader is let in immediately, even if a writer is already waiting its turn.

READER PREFERENCE VS. WRITER PREFERENCE — A TRADEOFF, NOT A BUG

A second variant flips the priority: once a writer is waiting, no new reader is allowed to start, so writers are never starved. Neither variant is universally “correct” — each protects one class of process at the cost of risking starvation for the other.

The Writer

```
int read_count = 0;          /* readers currently reading */
semaphore mutex  = 1;        /* protects read_count itself */
semaphore rw_mutex = 1;      /* exclusive access for a writer */

/* Writer */
do {
    wait(rw_mutex);
    /* writing is performed */
    signal(rw_mutex);
} while (true);
```

The Reader

```
/* Reader */
do {
    wait(mutex);
    read_count++;
    if (read_count == 1) { wait(rw_mutex); } /* first reader locks out writers */
    signal(mutex);
    /* reading is performed */
    wait(mutex);
    read_count--;
    if (read_count == 0) { signal(rw_mutex); } /* last reader lets a writer in */
    signal(mutex);
} while (true);
```

Only the First In, Only the Last Out

- `read_count` is itself shared data, incremented and decremented by potentially many readers concurrently, so it needs its own protection — mutex — or updating it would reproduce Lecture 13's race condition.
- Only the first reader to arrive (the one that pushes `read_count` from 0 to 1) actually acquires `rw_mutex`, locking out writers on behalf of every reader that follows.
- Every subsequent concurrent reader simply increments `read_count` and goes straight to reading — it never touches `rw_mutex` at all, which is exactly what allows all of them to read genuinely in parallel.
- Symmetrically, only the last reader to leave (the one that brings `read_count` back to 0) releases `rw_mutex`, re-opening the door for a waiting writer.

Key Takeaways

- The bounded-buffer problem needs three semaphores doing three separate jobs: empty and full (counting semaphores tracking free and filled slots) and mutex (protecting the shared buffer index).
- Order matters: a counting semaphore (empty/full) must always be acquired before mutex, never after — reversing the order lets a blocked process hold mutex forever, deadlocking the system.
- The first readers-writers problem gives priority to readers: a reader only ever waits if a writer already has the data, which can starve a writer under a steady stream of readers.
- The semaphore solution protects `read_count` with its own mutex, and uses `rw_mutex` as the single lock that only the first arriving reader acquires and only the last departing reader releases.